

Rapport de Projet Data Engineering

Industrialisation d'un Pipeline E-commerce avec Git, Jenkins, Apache Airflow et MongoDB

Réalisé par : Boyédé ADEOTCHOUN

1. Introduction et contexte métier

Dans un contexte de transformation numérique, les entreprises génèrent quotidiennement de très grands volumes de données. Dans le secteur du commerce électronique, ces données concernent notamment les commandes, les clients, les produits et les ventes. Leur exploitation permet de suivre les performances commerciales, d'analyser le comportement des clients et d'aider les décideurs à orienter leur stratégie.

Cependant, lorsque ces traitements sont réalisés manuellement, plusieurs difficultés apparaissent :

- production lente des rapports ;
- risque important d'erreurs humaines ;
- absence d'automatisation ;
- manque de traçabilité ;
- difficulté à conserver un historique des indicateurs.

L'entreprise étudiée dans ce projet est spécialisée dans la vente de produits informatiques. Jusqu'à présent, les données étaient exportées sous forme de fichiers CSV puis traitées manuellement afin de produire différents indicateurs de performance. Cette méthode demandait un temps important et ne garantissait pas une qualité constante des résultats.

La problématique du projet est donc la suivante :

Comment automatiser l'ensemble de la chaîne de traitement des données afin de produire rapidement et de manière fiable les indicateurs nécessaires au pilotage de l'activité ?

Pour répondre à cette problématique, une architecture de Data Engineering a été développée en utilisant plusieurs outils complémentaires :

- Git pour la gestion des versions ;
- Jenkins pour l'intégration continue ;
- Apache Airflow pour l'orchestration des traitements ;
- Python pour les traitements ETL ;
- MongoDB pour le stockage des indicateurs ;
- Docker pour la conteneurisation de l'ensemble des services.

L'objectif est de construire un pipeline automatisé capable de collecter, nettoyer, transformer, analyser et stocker les données de ventes tout en assurant la qualité des traitements.

2. Présentation du dataset

Le projet utilise le Olist Brazilian E-Commerce Dataset, un jeu de données public contenant plus de 100 000 commandes réelles réalisées sur une plateforme e-commerce brésilienne.

Les principaux fichiers exploités sont :

- olist_orders_dataset.csv
- olist_order_items_dataset.csv
- olist_customers_dataset.csv
- olist_products_dataset.csv
- product_category_name_translation.csv

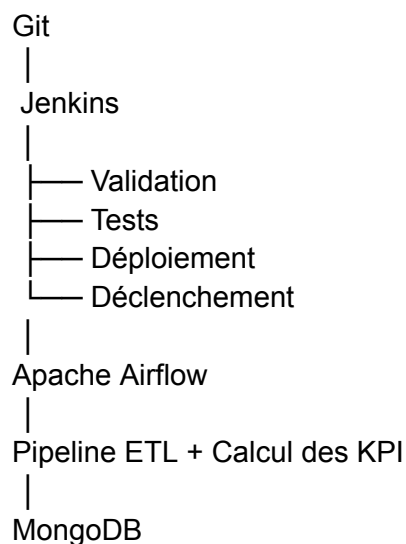
Ces fichiers permettent d'obtenir des informations sur :

- les commandes ;
- les clients ;
- les produits ;
- les catégories ;
- les prix ;
- les régions ;
- les dates d'achat.

Ces différentes sources sont fusionnées afin de construire un jeu de données unique utilisé pour les traitements analytiques.

3. Architecture technique

L'architecture repose sur plusieurs composants travaillant ensemble.



Chaque composant possède un rôle précis.

Git

Git permet de gérer les versions du projet, de suivre les modifications du code et de faciliter le développement collaboratif.

Jenkins

Jenkins automatise les différentes étapes du projet grâce à une pipeline CI/CD. À chaque exécution, il lance les tests, vérifie la validité du projet puis déclenche automatiquement le pipeline Airflow.

Apache Airflow

Airflow orchestre l'ensemble des traitements de données. Il exécute les différentes tâches dans le bon ordre, gère les dépendances entre elles et assure le suivi des exécutions.

MongoDB

MongoDB stocke les indicateurs calculés après chaque exécution afin de conserver un historique des résultats.

Docker

Docker permet d'exécuter tous les services dans des conteneurs indépendants, garantissant ainsi un environnement identique sur toutes les machines.

PostgreSQL

PostgreSQL est utilisé par Airflow pour stocker les métadonnées des DAG, des tâches et des exécutions.

4. Développement du pipeline ETL

Le pipeline ETL a été développé en Python.

Les données sont chargées depuis les fichiers CSV présents dans le dossier data/raw.

Les différentes tables du dataset Olist sont fusionnées afin d'obtenir un dataset unique contenant :

- les commandes
- les clients
- les produits
- les catégories

Plusieurs règles de validation sont appliquées avant les traitements :

- montant supérieur à zéro
- quantité positive
- fichier non vide
- détection des données invalides

Les données valides sont enregistrées dans dataset.csv, tandis que les lignes rejetées sont sauvegardées dans errors.csv.

Cette étape garantit la qualité des données avant le calcul des indicateurs.

5. Calcul des KPI

Une fois les données nettoyées, le script calculate_kpis.py calcule automatiquement plusieurs indicateurs métier.

Les KPI produits sont notamment :

- nombre total de commandes : 98 666
- nombre total de clients : 95 420
- chiffre d'affaires total : 15 843 553,24
- panier moyen : 160,58
- nombre de lignes valides : 112 650
- nombre de lignes rejetées : 0
- Top 10 des produits
- chiffre d'affaires par catégorie
- chiffre d'affaires par région

Les résultats sont enregistrés dans le fichier kpis.json.

6. Stockage des indicateurs dans MongoDB

Les KPI calculés sont enregistrés dans la base de données ecommerce_analytics, au sein de la collection sales_metrics.

Chaque exécution ajoute un nouveau document contenant :

- la date d'exécution ;
- le statut du pipeline ;
- les KPI calculés ;
- les indicateurs régionaux ;
- les indicateurs par catégorie.

Cette approche permet de conserver un historique complet des analyses réalisées.

```
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

The server generated these startup warnings when booting
2026-07-06T18:58:50.813+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-07-06T18:58:51.265+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-07-06T18:58:51.265+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the following sysfsFile

test> use ecommerce_analytics
switched to db ecommerce_analytics
ecommerce_analytics> db.sales_metrics.find().pretty()
{
  _id: ObjectId('6a4bf25bb7b5aa54ee5fec842'),
  execution_date: '2026-07-06T20:22:19.452328',
  dag_id: 'ecommerce_sales_pipeline',
  dataset: 'olist',
  status: 'success',
  global_metrics: {
    nb_commandes: 98466,
    nb_clients: 95420,
    chiffre_affaires: 15843553.24,
    panier_moyen: 160.58
  },
  top_products: {
    bb50f2e236e5eea0180680137654686c: 67606.1,
    d1c427060a8f73f0b889a5c7c61f2a4: 68976.03,
    c6cd53943a98f92890844667889f1890: 59893.99,
    '99a4788cb24856965c36a24e339b6858': 51871.6,
    d6160fb7873f184099d9bc95e38376af: 50326.18,
    '30d2a17168ec895c781a9191c1e95a07': 48212.22,
    ace2eb7d00ea1a7bdebd4e68314663af: 44828.76,
    '5f504b3a1c75b73d6151be81ab88bdc9': 41725.81,
    '25c38557cf79387ec5abdd5931f922db': 40311.95,
    '53b36df67ebb7c41585e8d54d6772e08': 39957.93
  },
  ca_region: {
    AC: 19669.7,
    AL: 96229.4,
    AM: 27835.73,
    AP: 16262.8,
    BA: 611586.67,
    CE: 275606.3,
    DF: 353229.44,
    ES: 324801.91,
    GO: 347706.93,
    MA: 151171.99,
    MG: 1856161.49,
    MS: 135986.67,
    MT: 186168.96,
    PA: 217647.11,
    PB: 148987.81,
    PE: 322237.69,
    PI: 100132.08,
    PR: 800935.44,
    RJ: 2129681.98,
    RN: 101895.08,
    RO: 57058.02,
  }
}
```

7. Orchestration avec Apache Airflow

Le pipeline est orchestré grâce au DAG ecommerce_sales_pipeline.

Le DAG commence par vérifier la présence du fichier CSV à l'aide d'un FileSensor. Une fois le fichier détecté, différentes vérifications sont effectuées afin de s'assurer qu'il n'est pas vide et qu'il peut être traité.

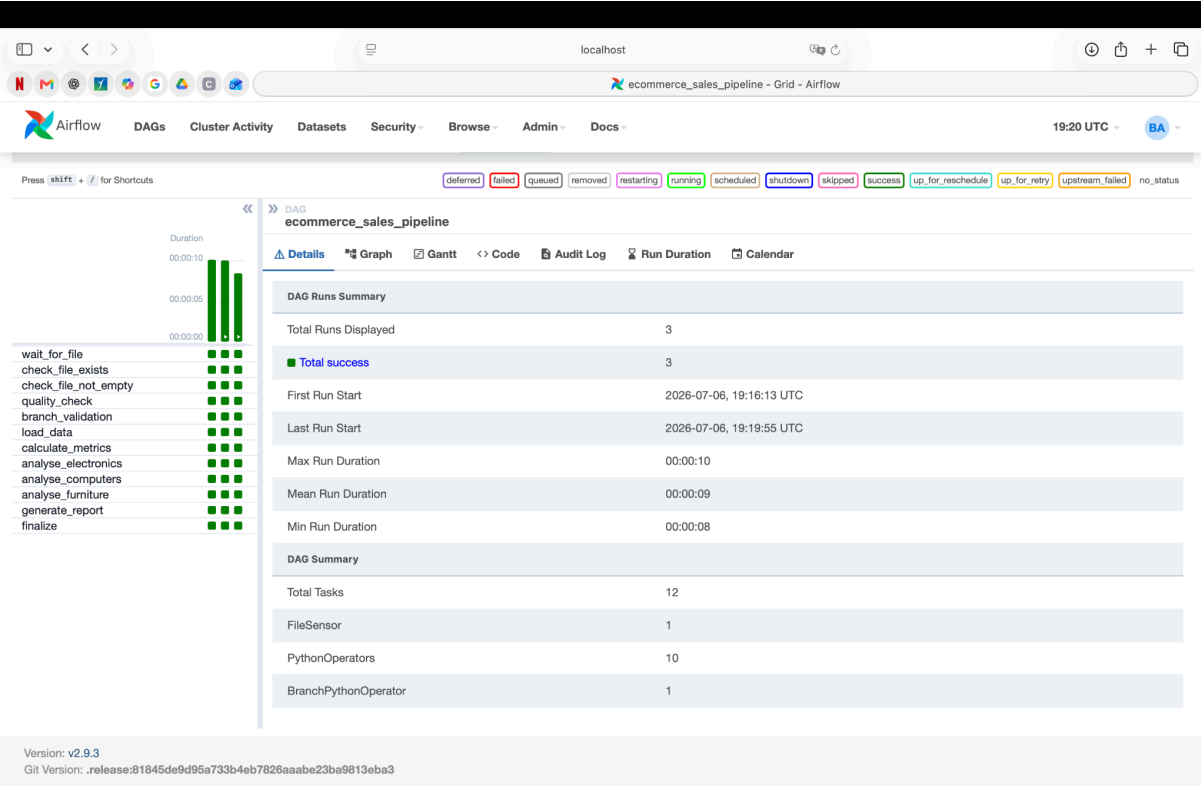
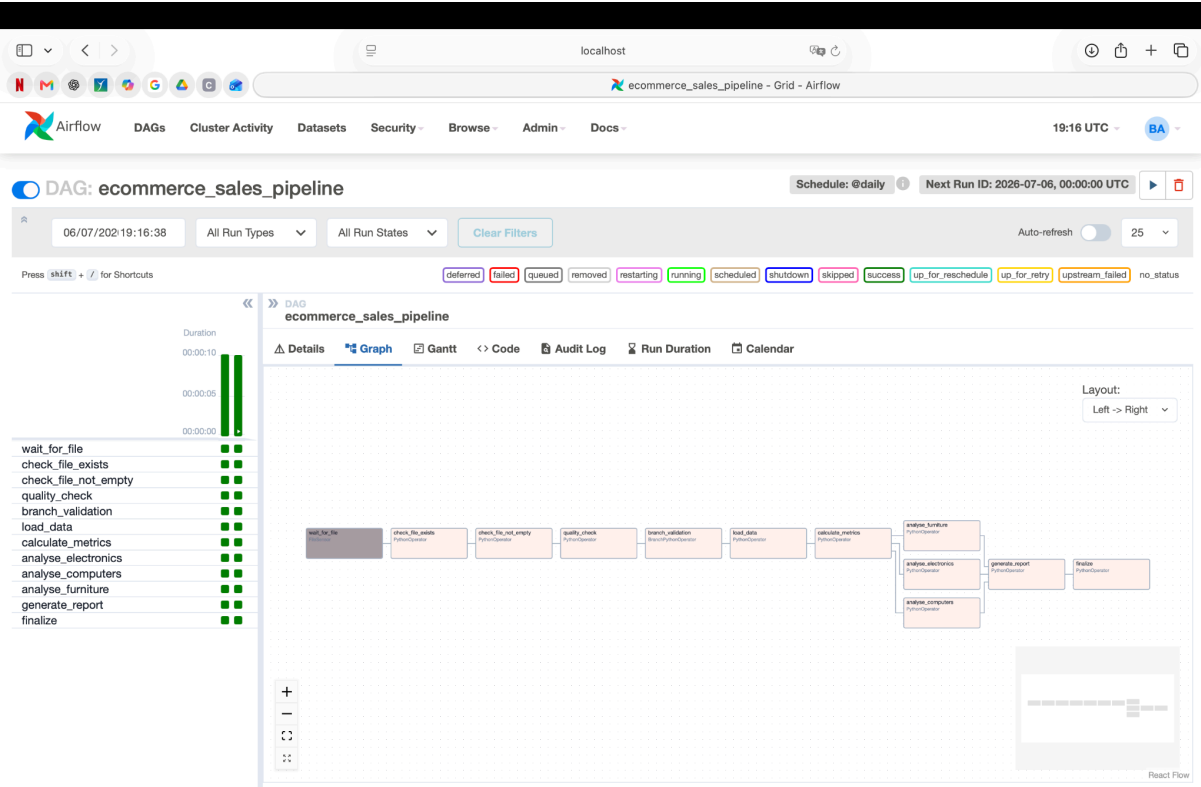
Le pipeline ETL est ensuite exécuté afin de nettoyer et transformer les données.

Les KPI sont calculés automatiquement puis enregistrés dans MongoDB.

Les principales fonctionnalités utilisées sont :

- FileSensor pour attendre l'arrivée des données ;
- BranchPythonOperator afin de choisir le chemin d'exécution selon les contrôles réalisés ;
- XCom pour transmettre des informations entre les différentes tâches ;
- tâches dynamiques pour générer automatiquement plusieurs traitements selon les catégories de produits ;
- TriggerRule.ALL_DONE afin d'assurer la poursuite du pipeline même lorsqu'une tâche rencontre une erreur.

À la fin de chaque exécution, un rapport automatique est généré dans le fichier report.txt.



8. Intégration continue avec Jenkins

Le projet utilise Jenkins afin d'automatiser le processus d'intégration continue.

Le pipeline est composé des étapes suivantes :

Checkout

Récupération du projet depuis le dépôt Git.

Install Dependencies

Installation automatique des bibliothèques Python nécessaires au projet.

Run Tests

Exécution des tests unitaires développés avec Pytest.

Validate DAG

Vérification de la validité du DAG Airflow avant son déploiement.

Deploy DAG

Copie automatique du DAG dans le dossier utilisé par Airflow.

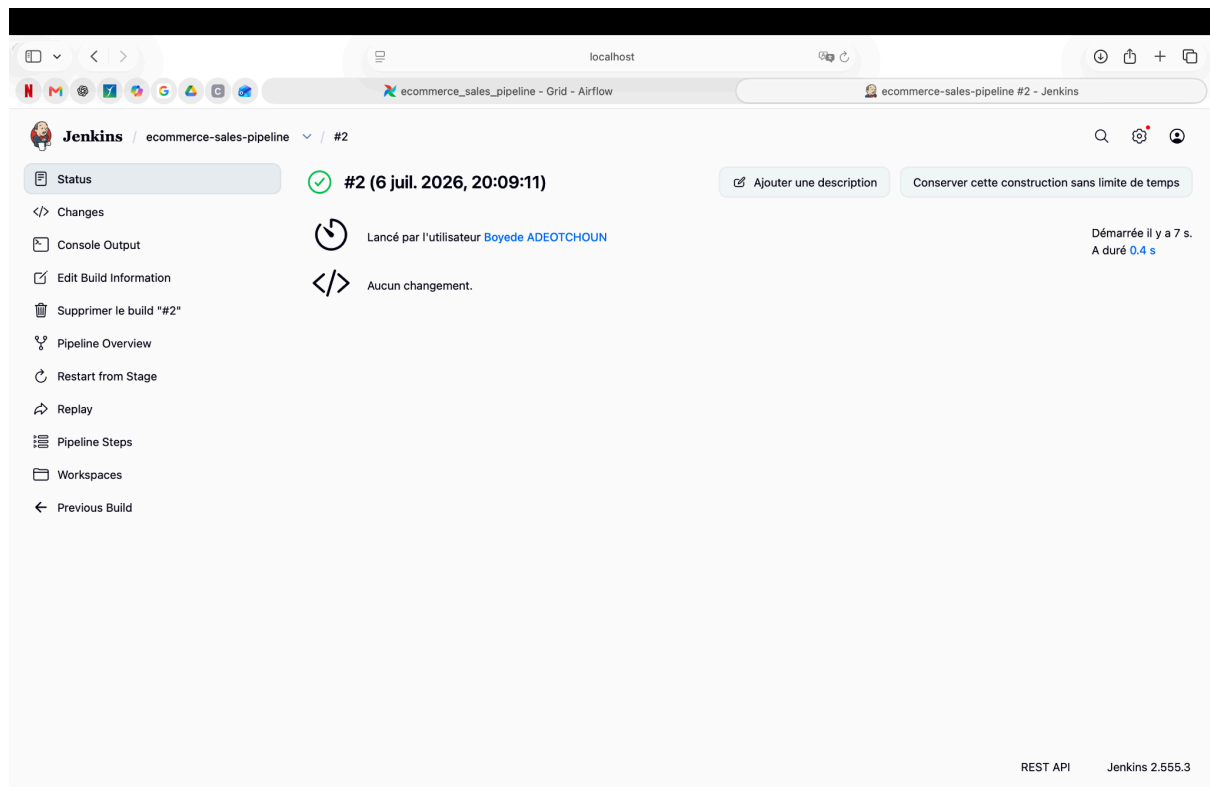
Trigger DAG

Déclenchement automatique du pipeline Airflow.

Verify MongoDB

Vérification que les KPI ont bien été enregistrés dans MongoDB.

L'automatisation de ces étapes garantit un déploiement fiable et reproductible du projet.



9. Gestion du code source avec Git

Le développement du projet a été réalisé avec Git en utilisant :

- une branche principale `main` ;
- une branche de développement `dev` ;
- plusieurs commits permettant de suivre l'évolution du projet ;
- une fusion finale de la branche `dev` vers la branche `main`.

L'utilisation de Git a permis d'assurer la traçabilité des modifications et une meilleure organisation du développement.

```
report_task >> final_task
online
76459e2 (HEAD -> main) Update project documentation
0ef95f2 Update Jenkins and Docker configuration
974e082 (dev) Add gitignore
776d5e0 Add MongoDB verification script
6a4203a Add Jenkins pipeline
a610465 Add Pytest tests
8135e08 Add advanced Airflow workflow
ccb4091 Add Airflow DAG skeleton
2e1f9a0 Add Airflow Docker services
4e0e72c Add MongoDB integration
1a95d25 Add KPI calculation script
6d474e8 Add ETL processing script
c762086 Add Olist dataset
18b1eb8 Add project dependencies
224003e Initial project structure
(venv) adeotchounfrida@MacBook-Air-de-ADEOTCHOUN airflow-ecommerce-project % git log --
```

Au total, plus de dix commits ont été réalisés durant le développement du projet, couvrant les différentes étapes : préparation du projet, ETL, calcul des KPI, intégration MongoDB, développement du DAG Airflow, tests Pytest, Jenkins et documentation finale.

10. Tests réalisés

Les tests unitaires ont été développés avec le framework Pytest.

Les principaux tests portent sur :

- l'existence du dataset ;
- la validité des montants ;
- la validité des quantités ;
- la validation des commandes.

Les quatre tests ont été exécutés avec succès.

```
(venv) adeotchounfrida@MacBook-Air-de-ADEOTCHOUN airflow-ecommerce-project % pytest tests/
===== test session starts =====
platform darwin -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/adeotchounfrida/airflow-ecommerce-project
collected 4 items

tests/test_pipeline.py .... [100%]

===== 4 passed in 1.58s =====
(venv) adeotchounfrida@MacBook-Air-de-ADEOTCHOUN airflow-ecommerce-project % docker exec -it mongo01 mongo
```

11. Résultats obtenus

Le projet a permis :

- l'automatisation complète de la chaîne de traitement des données ;
- le calcul automatique des KPI ;
- l'orchestration des traitements avec Apache Airflow ;
- le stockage des résultats dans MongoDB ;
- la mise en œuvre d'une pipeline CI/CD avec Jenkins ;
- le contrôle automatique de la qualité des données ;
- l'historisation des indicateurs.

L'ensemble du processus est désormais reproductible, fiable et entièrement automatisé.

12. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du développement :

- configuration des conteneurs Docker ;
- installation et configuration de MongoDB ;
- initialisation d'Apache Airflow ;
- gestion des dépendances Python ;
- configuration de Jenkins ;
- intégration entre les différents outils.

Ces difficultés ont été progressivement résolues grâce à des phases successives de tests et de validation.

13. Conclusion

Ce projet a permis de concevoir une solution complète de Data Engineering répondant aux besoins d'une plateforme e-commerce.

Grâce à l'utilisation conjointe de Git, Docker, Jenkins, Apache Airflow, Python et MongoDB, il a été possible d'automatiser l'ensemble du cycle de traitement des données, depuis leur ingestion jusqu'au stockage des indicateurs métier.

Les objectifs fixés au début du projet ont été atteints :

- automatisation des traitements ;
- amélioration de la qualité des données ;
- historisation des KPI ;
- mise en place d'une architecture moderne de Data Engineering.